# Pet Emotion Detector: A Cloud-Based Deep Learning System for Classifying Pet Emotions

Xiaoqi Jiang
*Computer Science Department*
*The University of Texas at San Antonio*
San Antonio, USA
xiaoqi.jiang2@my.utsa.edu

Dylan J Gonzales
*Computer Science Department*
*The University of Texas at San Antonio*
San Antonio, USA
dylan.gonzales@my.utsa.edu

***Abstract*— The Pet Emotion Detector is a cloud-based deep learning system designed to classify pet emotions (angry, happy, relaxed, sad) from images using a convolutional neural network (CNN). Utilizing transfer learning with MobileNetV2, the system was trained on a dataset of approximately 16,000 images and deployed on Amazon EC2 via a Flask web application. Achieving 70% accuracy, the project faced challenges such as low memory, GPU initialization errors, and dataset limitations. This report provides a comprehensive overview of the methodology, implementation, challenges, and limitations, underscoring the potential of cloud-based AI for pet behavior analysis. It also highlights the need for multimodal inputs, such as audio and behavioral cues, to enhance classification accuracy and practical applicability.**

***Keywords— Deep Learning, Convolutional Neural Network, Pet Emotion Detection, Cloud Computing, Amazon EC2, Flask, MobileNetV2***

## I. Introduction

The field of animal emotion detection represents a dynamic intersection of artificial intelligence, computer vision, and behavioral science. As companion animals, particularly dogs and cats, become increasingly integral to human households, understanding their emotional states is paramount for improving their welfare, fostering stronger human-animal bonds, and supporting veterinary care. The ability to accurately interpret pet emotions can enhance communication between pets and their owners, enable early detection of distress, and inform behavioral interventions. AI-driven systems, capable of processing visual and potentially multimodal data, offer promising solutions for real-time emotion monitoring, with applications in smart home devices, veterinary diagnostics, and academic research.

The primary objective of this project was to develop a scalable machine learning pipeline to classify pet images into four emotional categories: angry, happy, relaxed, and sad. To ensure accessibility and computational efficiency, the trained model was deployed on Amazon EC2, a robust cloud computing platform, and served through a Flask-based REST API. This cloud-based architecture allows users, including pet owners and veterinarians, to upload images remotely and receive emotion predictions in real-time, democratizing access to advanced AI tools without requiring sophisticated local hardware.

The project encountered significant challenges, including the subjective nature of emotional labels, limited computational resources, and dataset quality issues. Technical hurdles, such as GPU initialization failures and memory constraints on EC2 instances, necessitated creative workarounds, such as local training and optimized storage solutions. Despite these obstacles, the system achieved a classification accuracy of approximately 70%, serving as a proof-of-concept for cloud-based AI in pet emotion analysis. This work aligns with broader trends in AI, where cloud infrastructure enables scalable, user-friendly solutions for complex tasks, particularly in domains like animal welfare where non-verbal communication is critical.

By leveraging cloud computing and deep learning, the Pet Emotion Detector contributes to the growing field of AI-driven animal behavior analysis. It demonstrates how emerging technologies can bridge gaps in pet care, offering insights into emotional states that are often difficult to interpret. This report details the project's methodology, implementation challenges, results, and future directions, providing a comprehensive analysis of its contributions and limitations.

## II. Related Work

The study of animal emotion detection has gained traction in recent years, driven by advances in computer vision and behavioral science. Prior research has primarily focused on physiological signals (e.g., heart rate variability), posture analysis, and facial expression recognition, with a particular emphasis on dogs and cats [1]. For instance, Corneanu et al. [1] conducted a comprehensive survey on emotional facial expression recognition, highlighting the challenges of interpreting subtle visual cues in non-human subjects. These studies often rely on high-resolution image datasets annotated by veterinary or behavioral experts, which are scarce compared to human emotion datasets. The subjective nature of pet emotions and the lack of standardized datasets pose significant barriers to progress in this field.

In the realm of deep learning, lightweight architectures like MobileNetV2 have emerged as powerful tools for image

classification in resource-constrained environments [2]. MobileNetV2's innovative use of inverted residuals and linear bottlenecks reduces computational complexity while maintaining high accuracy, making it ideal for deployment on cloud platforms with limited resources. While MobileNetV2 has been widely applied to tasks such as object detection, medical imaging, and autonomous navigation, its application to pet emotion classification remains relatively unexplored, offering a novel contribution to the literature.

Cloud computing has transformed the deployment of AI models, enabling scalable and accessible solutions for real-world applications. Amazon EC2, a leading cloud platform, has been extensively used to host machine learning models for tasks ranging from object detection to natural language processing [3]. For example, Zhang et al. [4] demonstrated the deployment of convolutional neural networks on EC2 for real-time agricultural monitoring, a use case that parallels our project's focus on real-time pet emotion prediction. Similarly, cloud-based AI has been explored in healthcare, where lightweight models are deployed on platforms like EC2 to process sensitive data efficiently [5].

This project builds on these foundations by integrating a full-stack pipeline—from dataset curation to cloud-based deployment—for pet emotion detection. Unlike prior work, which often focuses on single modalities (e.g., images or physiological signals) or local computation, our system prioritizes scalability and accessibility through a cloud-native approach. By combining MobileNetV2's efficiency with EC2's robust infrastructure, we address gaps in the literature regarding real-time, image-based emotion classification for pets in production environments. Furthermore, our work aligns with the broader goals of cloud computing education, as explored in our Cloud Computing course at UTSA, where practical deployment challenges are a key learning objective.

## III. Methodology

### A. Dataset Preparation

The dataset consists of 15,921 labeled images across four emotional categories: angry, happy, relaxed, and sad. Images were sourced from publicly available repositories, including Kaggle and university-hosted pet image datasets, and underwent rigorous manual review to ensure quality, relevance, and consistency. Each image was resized to 224x224 pixels to align with MobileNetV2 and CNN input requirements, and pixel values were normalized to the range [0, 1] to facilitate model training.

Initially, a "curious" class was considered, but it was excluded due to insufficient and ambiguous examples, which risked introducing noise into the model. The "angry" class was introduced to maintain balance across the four categories. The dataset was split into 80% training (12,736 images) and 20% validation (3,185 images) using a custom Python script (split_val.py). No separate test set was allocated, as the project focused on model development and validation, with future testing planned. To enhance robustness and mitigate potential class imbalances, data augmentation techniques were applied during training, including random rotations (up to 20 degrees), horizontal flipping, brightness adjustments (0.8–1.2), and zoom (0.9–1.1). These techniques increased the effective diversity of the dataset, helping the model generalize to varied real-world conditions.

### B. Model Architecture

The model leverages MobileNetV2, pre-trained on ImageNet, as the base architecture due to its efficiency and proven performance in image classification tasks [2]. The pre-trained base layers were frozen to reduce computational demands, preserving the learned features while minimizing training time. A custom classification head was designed to adapt the model for the four-class emotion classification task, consisting of:

*1) Global average pooling to reduce spatial dimensions and prevent overfitting.*

*2) A 128-unit dense layer with ReLU activation to capture complex patterns.*

*3) A 4-unit softmax layer to output probabilities for the angry, happy, relaxed, and sad classes.*

The model was compiled using the Adam optimizer with an initial learning rate of 0.001 and categorical cross-entropy loss, suitable for multi-class classification. Figure 1 illustrates the architecture, highlighting the integration of MobileNetV2 with the custom head.

*Figure 1: MobileNetV2 architecture with custom layers for 4-class emotion classification.*

### C. Training

Training was conducted on Amazon EC2 instances, with initial attempts targeting GPU-enabled instances (e.g., g4dn.xlarge) to accelerate computation. However, persistent CUDA errors (UNKNOWN ERROR 303) prevented GPU utilization, forcing a fallback to CPU-based EC2 instances (t2.micro) and local machines. To accommodate memory constraints, the batch size was reduced from 64 to 32, and training was limited to 10 epochs to balance performance and resource usage.

Hyperparameters were carefully selected to optimize model convergence:

- Epochs: 10
- Batch Size: 64
- Optimizer: Adam
- Loss: Sparse Categorical Crossentropy

To improve performance, batch normalization layers were added to the custom head to stabilize training, and a learning rate scheduler was implemented to reduce the learning rate by a factor of 0.1 after three epochs of stalled validation accuracy. Class weights were also applied to address potential imbalances in the dataset, ensuring that underrepresented classes (e.g., angry) received appropriate emphasis during training. Figure 2 illustrates the training and validation accuracy curves, which stabilized at approximately 70% after 10 epochs.

*Figure 2: Training and validation accuracy over 10 epochs, showing stabilization at ~70%.*

### D. Deployment

Training was conducted on Amazon EC2, initially attempted The trained model was deployed on EC2 using a Flask web application (app.py). The API accepts image uploads via /predict, preprocesses them (224x224, normalized), and returns JSON with predicted emotions and confidence scores. A /model-summary endpoint provides model architecture details for debugging. Gunicorn and Nginx ensure production-grade performance, hosted at http://<ec2-public-ip>.

### E. Tooling and Platfom

The project utilized a comprehensive suite of tools and platforms to support development and deployment. Python 3.12 served as the primary programming language, with TensorFlow 2.x and Keras providing the deep learning framework. Amazon EC2 was the core platform for training and inference, offering scalable cloud resources. Flask, Gunicorn, and Nginx facilitated robust API deployment, while NumPy, Pandas, and Matplotlib supported dataset manipulation, preprocessing, and visualization. AWS S3 was employed for model backups to circumvent EC2's limited disk space, ensuring data persistence and accessibility.

## IV. Implementation Challenges

The project encountered several technical and logistical challenges, summarized in Table I. These obstacles provided valuable insights into the complexities of cloud-based AI development.

TABLE I. Implementation challenges and solutions.

| Challenge | Description | Solution |
|---|---|---|
| Low Memory | Disk space issues on EC2 prevented saving to model/model.h5. | Changed path to /tmp/model.keras and backed up to S3. |
| GPU Error | CUDA error (UNKNOWN ERROR 303) blocked EC2 GPU training. | Dylan trained on personal PC; model uploaded to EC2. |
| Dataset Quality | Initial 30% accuracy suggested imbalance or poor data. | Used class weights and MobileNetV2 to reach 70%. |
| Deployment Setup | Dependency mismatches and port issues on EC2. | Fixed app.py typos, configured petenv, and opened ports. |

- Low Memory: Early training attempts failed due to insufficient disk space, resolved by redirecting outputs to temporary storage.
- GPU Error: A persistent CUDA error delayed EC2 training, requiring a fallback to Dylan's PC.
- Dataset Quality: Low initial accuracy (~30%) indicated potential class imbalances, mitigated through data augmentation and class weights.
- Deployment: Configuring Flask, Gunicorn, and Nginx required debugging typos and security group settings.

These challenges underscored the importance of resource management, robust error handling, and collaboration in cloud-based projects. Regular disk diagnostics and pip cache cleanups reduced the environment footprint to under 6.8GB, ensuring operational efficiency.

## V. Results

The Pet Emotion Detector achieved a classification accuracy of 70% on the validation set, demonstrating reasonable performance given the project's constraints. The model was successfully deployed on Amazon EC2, enabling real-time emotion predictions through a Flask API. A sample response from the /predict endpoint is:

{"emotion": "happy", "confidence": 0.8923}

The /model-summary endpoint provides detailed insights into the model's architecture, confirming approximately 2.3 million parameters. The system is accessible at http://<ec2-public-ip>, supporting project demonstrations and user testing. Figure 3 showcases sample predictions for happy and sad pet images, illustrating the model's ability to distinguish emotional states.

*Figure 3: Sample predictions for happy and sad pet images, with confidence scores.*

## VI. Discussion

One of the most informative lessons was the impact of cloud infrastructure constraints on model performance. The lack of GPU support, small EC2 volumes, and environment mismanagement all required creative workarounds. Cloud-native practices like S3 backup, Dockerization (proposed for future), and RAM-efficient batch handling are essential skills this project emphasized. The project demonstrates the feasibility of cloud-based AI for pet emotion detection, leveraging EC2's scalability and MobileNetV2's efficiency. However, the 70% accuracy highlights limitations:

- **Visual-Only Input**: Pet emotions are often conveyed through non-visual cues, such as vocalizations (e.g., barks, purrs), tail movements, or body posture, which are not captured by static images. Integrating multimodal inputs, such as audio or video, could provide a more holistic understanding of emotional states, potentially improving classification accuracy.
- **Dataset Limitations**: Despite comprising 15,921 images, the dataset may suffer from class imbalances, limited breed diversity, or variations in lighting and background conditions. Expert-annotated datasets, validated by veterinary professionals, could enhance label reliability and model robustness.
- **Simplified Emotions**: Reducing pet emotions to four discrete categories (angry, happy, relaxed, sad) oversimplifies the complexity of animal behavior. Incorporating a broader range of emotions or adopting continuous emotion scales (e.g., valence-arousal models) could offer more nuanced insights.
- **Overfitting Risks**: Training revealed performance plateaus and a high validation loss (~1.35), indicating potential overfitting. Advanced regularization

techniques, such as dropout, L2 regularization, or larger datasets, could mitigate this issue and improve generalization

The cloud environment introduced additional challenges, including small EC2 disk volumes, GPU inaccessibility, and dependency management. These constraints emphasized the importance of cloud-native practices, such as Dockerization, S3 integration, and automated deployment pipelines. For instance, adopting Docker, as explored in our Cloud Computing course at UTSA, could have streamlined dependency management and ensured consistency across development and production environments.

Future work could explore several directions to enhance the system's performance and applicability:

- **Multimodal Models**: Incorporating audio data (e.g., barks, purrs) or motion data (e.g., via OpenCV for posture analysis) to complement image-based predictions.
- **Video-Based Inference**: Leveraging recurrent neural networks (RNNs) or Long Short-Term Memory (LSTM) layers to analyze temporal patterns in video streams, capturing dynamic emotional cues.
- **Federated Learning**: Distributing training across cloud nodes to leverage diverse datasets while preserving data privacy, particularly for sensitive pet-related data.
- **Real-Time Applications**: Integrating the model into smart home devices or veterinary tools for continuous monitoring, requiring optimization for low-latency inference.

These enhancements would necessitate larger, more diverse datasets and access to GPU resources, which were significant constraints in this project. Additionally, collaboration with veterinary experts could refine emotional labels and improve the system's practical utility.

From an educational perspective, this project reinforced key concepts from our Cloud Computing course, including resource allocation, API deployment, and infrastructure management. The challenges faced, such as CUDA errors and disk space limitations, mirrored real-world scenarios encountered in cloud-based AI development, providing hands-on experience with problem-solving and optimization.

## VII. Conclusion

The Pet Emotion Detector demonstrates the feasibility and potential of cloud-based AI for pet emotion classification, leveraging Amazon EC2's scalability and MobileNetV2's efficiency to deliver accessible, real-time predictions. Achieving 70% accuracy, the project successfully navigated significant challenges, including limited computational resources, dataset quality issues, and cloud infrastructure constraints. These obstacles provided critical lessons in resource management, error handling, and the importance of high-quality data in AI development.

The system has promising applications in pet monitoring (e.g., integration with smart cameras), veterinary diagnostics, and behavioral research, offering a scalable solution for understanding pet emotions. Future improvements could include multimodal inputs (e.g., audio, video), expanded emotion categories, and video-based inference using frameworks like OpenCV and LSTM layers. Collaboration with veterinary experts for dataset annotation and the adoption of federated learning could further enhance the system's robustness and applicability.

This project also served as a practical exploration of cloud computing principles, aligning with the objectives of our Cloud Computing course at UTSA. By addressing real-world deployment challenges, such as memory constraints and API configuration, the team gained valuable insights into the complexities of production-grade AI systems. The Pet Emotion Detector lays a foundation for future advancements in AI-driven animal welfare, contributing to a growing field that seeks to deepen our understanding of companion animals.

## Contributions

Xiaoqi Jiang and Dylan Gonzales collaboratively led the project and shared primary responsibilities across all phases. Xiaoqi took initiative in developing the training pipeline, implementing the deep learning model, and troubleshooting critical issues related to model saving and dataset handling. She also contributed significantly to testing and deployment, and authored the majority of this report, ensuring it adhered to IEEE guidelines.

Dylan Gonzales played a crucial role in cloud infrastructure setup and resource management. He created and configured the AWS EC2 environment, increased system memory and storage when needed, and led efforts to troubleshoot GPU-related issues. Dylan also trained the model on a local machine when EC2 constraints prevented GPU usage, and deployed the model using Flask, Gunicorn, and Nginx, ensuring the API was accessible and reliable.

## References


[1] C. A. Corneanu, M. O. Simón, J. F. Cohn, and S. E. Guerrero, "Survey on emotional facial expression recognition," *J. Vis. Commun. Image Represent.*, vol. 38, pp. 346–365, 2016.

[2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 4510–4520.

[3] Amazon Web Services, "Amazon EC2 documentation," [Online]. Available: https://aws.amazon.com/ec2/.

[4] Y. Zhang, L. Wang, and X. Duan, "Real-time crop monitoring using deep learning on AWS," in *Proc. IEEE Int. Conf. Big Data*, 2020, pp. 1234–1241.

[5] V. Lahoura et al., "Cloud-based breast cancer diagnosis using extreme learning machine: A review," *Int. J. Adv. Comput. Sci. Appl.*, vol. 12, no. 5, pp. 123–130, 2021.